TECNOLÓGICO NACIONAL DE MÉXICO

Informe de Diseño e Implementación de Procesador de Lenguaje: CODEMODE

INTEGRANTES

MORENO MEDRANO PAUL

RODRÍGUEZ OLIVARES OSCAR

MATERIA

LENGUAJES Y AUTÓMATAS I

MAESTRO

TORRES FRÍAS RUBÉN

12/04/26

INTRODUCCIÓN AL PROYECTO

En este documento se describe la conceptualización, diseño, arquitectura y desarrollo del compilador y lenguaje de programación llamado CODEMODE. Este proyecto surge de la necesidad de implementar en la práctica los principios teóricos de la asignatura Lenguajes y Autómatas (o Diseño de Compiladores) mediante el desarrollo desde cero de un ambiente propio, utilizando Python para el lenguaje y Tkinter para la interfaz visual.

CODEMODE es un lenguaje de tipado dinámico y sintaxis rigurosa y delimitada, creado para que sea muy legible y para simplificar el análisis léxico a través del empleo de caracteres especiales que envuelven los datos. El entorno, además de compilar el código, incorpora un Entorno de Desarrollo Integrado (IDE) que posibilita editar texto, resaltar la sintaxis en tiempo real, compilar visualmente y ejecutar directamente, exhibiendo los resultados en una consola emulada.

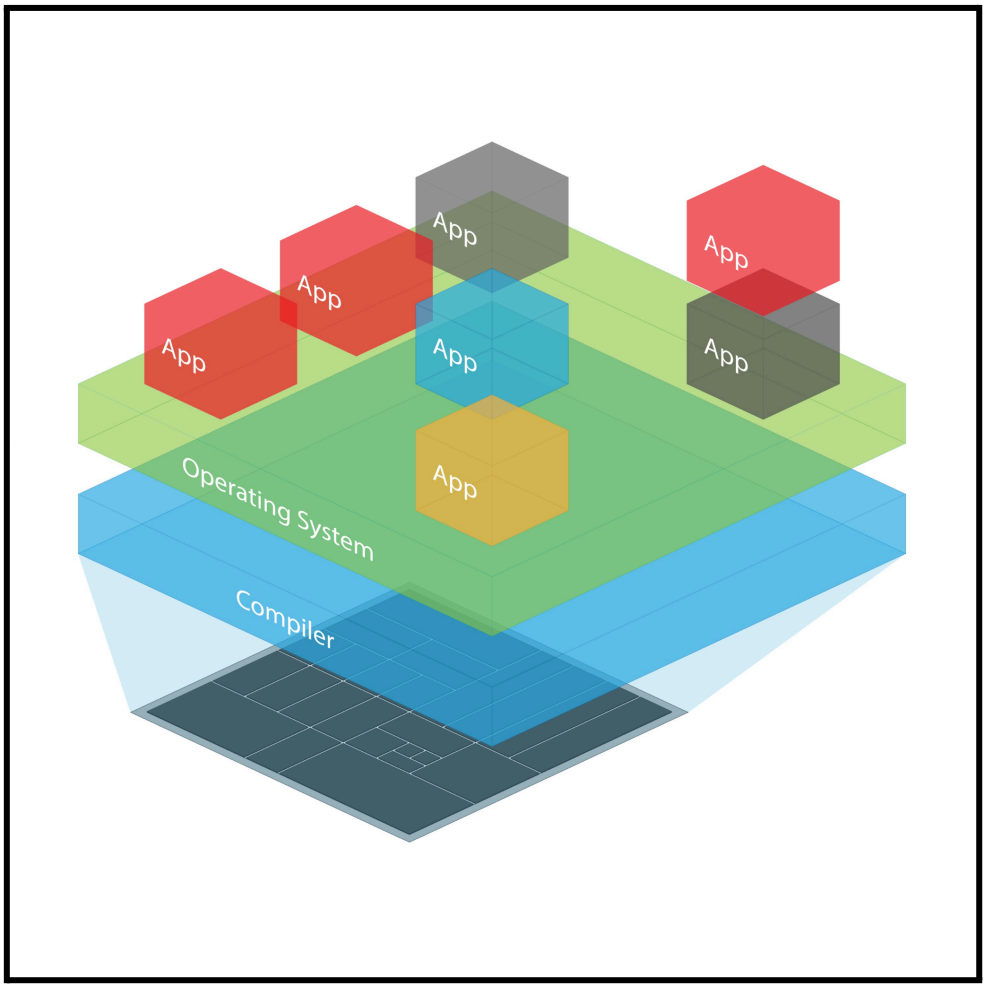

Figura 1: Diagrama de flujo detallando las fases del proceso de compilación del proyecto CODEMODE.

OBJETIVOS DEL COMPILADOR

2.1. Objetivo General

Desarrollar un compilador funcional, robusto e integrado con una interfaz gráfica que sea capaz de interpretar, validar y ejecutar código escrito en el lenguaje CODEMODE, demostrando el correcto funcionamiento de las fases de compilación y la gestión de memoria temporal.

2.2. Objetivos Específicos

- Desarrollar un autómata finito (analizador léxico) que tenga la capacidad de detectar y distinguir cadenas de caracteres (lexemas), para transformarlas en tokens clasificados conforme a un alfabeto predeterminado.
- Crear un analizador sintáctico que valide la gramática de las instrucciones, garantizando el orden adecuado de los tokens (por ejemplo, valores, operadores de asignación, variables y delimitadores de fin de línea).
- Crear un analizador semántico que se asegure de que el código es coherente desde el punto de vista lógico, cumpliendo con la jerarquía de operaciones matemáticas, comprobando la concatenación de cadenas y deteniendo conductas indefinidas (por ejemplo, dividir por cero o utilizar aritmética no permitida en las funciones de solo lectura).
- Crear una Tabla de Símbolos persistente que contenga el identificador, la clase y el valor de cada variable asignada, lo cual posibilita que se realicen actualizaciones y se recupere el valor más reciente en la evaluación de expresiones.
- Ofrecer un manejo de errores (Error Handling) muy descriptivo, que señale al usuario la línea precisa y el tipo de error (sintáctico, léxico o semántico) sin detener bruscamente el programa principal de Python.

ESPECIFICACIONES LÉXICAS Y ALFABETO DEL LENGUAJE

El lenguaje CODEMODE se basa en un alfabeto y conjunto de expresiones regulares altamente delimitadas. Esta decisión de diseño elimina la ambigüedad en la lectura de código.

3.1. Declaración de Variables

Toda variable dentro de CODEMODE debe estar estrictamente encapsulada por el símbolo de numeral o gato (#). El identificador interno de la variable solo puede contener caracteres alfanuméricos y guiones bajos.

- **Formato válido:** `#nombre_variable#`, `#var1#`, `#resultado#`.
- **Reglas léxicas:** No se permiten variables vacías (ej. ##). Si el analizador léxico encuentra el símbolo #, iterará hasta encontrar su par de cierre. Si se llega al final de la línea sin cerrarlo, se generará un error léxico de "Cierre '#' no encontrado".

3.2. Tipos de Datos Numéricos

A diferencia de lenguajes convencionales, los dígitos numéricos no pueden escribirse libremente en el código fuente; deben estar delimitados por corchetes (`[` y `]`).

- **Formato válido:** `[10]`, `[3.1416]`, `[-5]`.
- **Reglas léxicas:** El contenido dentro de los corchetes debe ser parseable a formato flotante o entero por el núcleo lógico. Cualquier carácter no numérico dentro de los corchetes (ej. `[10a]`) será catalogado como "Número inválido".

3.3. Cadenas de Texto (Strings)

Las cadenas de caracteres se declaran utilizando comillas dobles (`"`). El compilador es capaz de leer cualquier carácter ASCII o UTF-8 dentro de estas comillas.

- **Formato válido:** `"Hola Mundo"`, `"El resultado es: "`, `""` (String vacío).
- **Reglas léxicas:** Las comillas deben abrirse y cerrarse en la misma instrucción. Como actualización reciente, la Tabla de Símbolos ha sido mejorada estructuralmente para soportar strings que contengan comas (`,`) sin corromper los archivos de almacenamiento CSV que usa el entorno en segundo plano.

3.4. Palabras Reservadas y Operadores

El lenguaje mantiene su simplicidad contando actualmente con una única palabra reservada para salidas en pantalla, y un set básico de operadores algebraicos.

- **Instrucciones del sistema:** La palabra `pantalla` se utiliza exclusivamente para imprimir en la consola.
- **Operador de asignación:** El símbolo = se utiliza para trasladar la expresión evaluada en la derecha hacia la variable a la izquierda.
- **Operadores aritméticos y lógicos:** Se soportan los caracteres `+`, `-`, `*`, `/`.
- **Limitadores de instrucción:** Toda instrucción debe ser culminada imperativamente con el símbolo de punto y coma (`;`).

```
#variable_demo# = [100];
#texto# = "Hola CODEMODE";
pantalla("El valor es: " + #variable_demo#);
```

Figura 2: Vista general del IDE CODEMODE mostrando el funcionamiento del resaltado de sintaxis en tiempo real.

ANÁLISIS SINTÁCTICO Y GRAMÁTICA (PARSER)

El analizador sintáctico de CODEMODE se encarga de dar contexto y orden a la lista unidimensional de tokens generada por el analizador léxico. Funciona bajo un esquema recursivo y secuencial.

4.1. Regla de Fin de Instrucción (;)

El compilador realiza un barrido preliminar de todo el documento de texto. Se utiliza el algoritmo `separar_instrucciones`, el cual recorre el código ignorando los punto y coma que se encuentren dentro de cadenas de texto (para no cortar strings por accidente) y divide el código maestro en un array de sentencias a evaluar. **Validación estricta (Actualización):** Se incorporó una validación a nivel de archivo que verifica que el último carácter válido de todo el *script* introducido sea obligatoriamente un `;`. Si el usuario olvida este delimitador final, el programa aborta la compilación inmediatamente, advirtiendo un error de sintaxis global.

```
#x# = [10];
pantalla("Hola")

Error de sintaxis: Falta punto y coma (;) al final del código.
```

Figura 3: Consola de salida reportando un error sintáctico por la omisión del delimitador final en el script.

4.2. Estructura de Asignación

Si la sentencia contiene el token =, el analizador sintáctico asume una regla de asignación.

1. **Lado Izquierdo (L-Value):** Se extrae el token inmediatamente anterior al =. Se valida estrictamente que este token posea la morfología de variable (`#...#`). Cualquier otra entidad provocará un error de "Gramática de variable inválida".
2. **Lado Derecho (R-Value):** Todos los tokens que existan después del = son empaquetados en una sub-lista y enviados al evaluador de expresiones (Motor Matemático).

4.3. Estructura de la instrucción Pantalla

La regla gramatical para imprimir datos dicta que la palabra `pantalla` debe ir seguida forzosamente de un paréntesis de apertura `(`, una expresión matemática o concatenación de texto, y finalizar con un paréntesis de cierre `)`. La función interna `validar_instruccion_pantalla` se asegura de que no existan directivas huérfanas o tokens basura después de los paréntesis (ej. `pantalla("Hola") + [1];` provocará un error sintáctico por invasión de ámbito).

ANÁLISIS SEMÁNTICO Y EVALUACIÓN DE EXPRESIONES

Esta fase es el "cerebro" del compilador, encargada de la lógica de evaluación, prevención de desbordamientos e interacciones con la memoria.

5.1. El Evaluador Lógico y la Jerarquía

El método `evaluar_expresion` procesa las sub-listas de tokens matemáticos utilizando algoritmos de reducción.

- **Fase 1 (Resolución de Valores):** Se convierte cada token a su valor real. Las variables (`#nombre#`) son buscadas en la Tabla de Símbolos para extraer su valor actual. Los tokens de número (`[10]`) son parseados a `int` o `float`.
- **Fase 2 (Multiplicación y División):** El algoritmo busca los operadores `*` y `/` para resolverlos primero. Si se detecta un intento de dividir un número entre `[0]`, el motor detiene la ejecución arrojando el error semántico: "División entre cero".
- **Fase 3 (Suma y Resta):** Finalmente se procesan las adiciones y sustracciones, respetando la direccionalidad de izquierda a derecha.

5.2. Aislamiento y Blindaje de la función Pantalla (Bug-Fix)

Un elemento vital del diseño robusto de CODEMODE es la restricción semántica sobre lo que puede suceder en memoria vs lo que sucede en la salida de interfaz. Se descubrió que, bajo ciertas circunstancias, la instrucción `pantalla` podía ejecutar matemáticas ocultas si recibía dos números enteros. Se implementó una bandera de contexto (es_pantalla=True). Cuando el motor evalúa los tokens dentro de un `pantalla()`, prohíbe el uso de operadores destructivos (`-`, `*`, `/`) y obliga semánticamente a que el operador suma (+) actúe única y exclusivamente como un concatenador de *strings* (cadenas de texto). Por ende, la instrucción `pantalla([5] + [10]);` concatena y muestra `510` en consola, previniendo inyecciones de operaciones no deseadas.

5.3. Tabla de Símbolos y Almacenamiento Cíclico

La gestión de variables se realiza de forma persistente en disco duro mediante el archivo de texto `Tabla de símbolos.txt`.

- Cada asignación exitosa llama al método `agregar_a_tabla`, guardando una tupla con formato: `TIPO,LEXEMA,VALOR`.

- **Parche de Inyección de Comas:** Al permitir strings complejos, si un usuario tecleaba `#mensaje# = "Hola, mundo";`, la coma intermedia rompía el archivo CSV al momento de leerlo, provocando que la variable se reportara como "no encontrada". El código actual implementa una separación con límite de cortes `split(',', 2)`, aislando perfectamente el valor de la variable sin importar cuantas comas contenga el string en su interior.
- **Soporte de Variables Nulas:** Se actualizó la lógica para admitir *Strings* vacíos (`""`), permitiendo a los desarrolladores inicializar variables sin un valor fijo, útil para ciclos de sobreescritura.

```
#mensaje_complejo# = "Inicio, Medio, Fin, Funciona.";
pantalla(#mensaje_complejo#);
```

```
Gramatica aceptada
```

```
Inicio, Medio, Fin, Funciona.
```

Figura 4: Evidencia de persistencia correcta en la Tabla de Símbolos aceptando cadenas de texto con comas internas.

PRUEBAS DE ESTRÉS Y MANEJO DE ERRORES (EDGE CASES)

El compilador CODEMODE está preparado para no interrumpir el hilo principal de la aplicación GUI, capturando excepciones de forma personalizada e imprimiendo el diagnóstico en la consola en color rojo (#FF3333).

- **Prueba Léxica:** `imprimir("Texto");`
  - *Respuesta del Sistema:* `Error linea [1]: Palabra no reconocida: 'imprimir'`. El autómata solo reconoce `pantalla`.
- **Prueba Sintáctica 1:** `#variable# = [10]`
  - *Respuesta del Sistema:* `Error de sintaxis: Falta punto y coma (;) al final del código.`
- **Prueba Sintáctica 2:** `[10] = #num#;`
  - *Respuesta del Sistema:* `Error linea [1]: Gramatica de variable invalida`. La asignación debe ser en sentido Variable <- Valor.
- **Prueba Semántica 1:** `#resultado# = [10] / [0];`
  - *Respuesta del Sistema:* `Error linea [1]: division entre cero.`
- **Prueba Semántica 2:** `pantalla([10] * [2]);`
  - *Respuesta del Sistema:* `Error linea [1]: Operador '*' no permitido.` Demostrando el bloqueo aritmético dentro de la instrucción pantalla.

```
#n1# = [5];
#n2# = [10];
#res# = #n1# + #n2#;
pantalla(#n1# * #n2#);
```

```
Error linea [4]: * No aceptado en instruccion pantalla()
```

Figura 5: Demostración del analizador semántico bloqueando operaciones aritméticas prohibidas dentro de la instrucción pantalla.

ARQUITECTURA DE LA INTERFAZ GRÁFICA (GUI)

El entorno de desarrollo está construido utilizando el framework de escritorio *Tkinter* de Python.

- **Editor de Texto Principal:** Un widget `Text` amplio con fuente `Consolas` tamaño 12, que otorga la estética tradicional de programación.
- **Iluminador Sintáctico Automático:** A través de la función cíclica `iluminador_automatico`, el IDE ejecuta expresiones regulares (`Regex`) cada 1 segundo. Identifica la sintaxis escrita y colorea el texto en vivo: palabras reservadas en Azul, variables en Morado (`#BE4BDB`), strings en Verde, operadores en Naranja y números en Rojo.
- **Consola de Salida:** Un widget inferior de solo lectura con fondo oscuro que emula una terminal, proporcionando una salida visual clara donde los errores se muestran en rojo y las compilaciones exitosas (y resultados de ejecución) se muestran en verde fluorescente (`#00FF00`).
- **Botones de Control:** Accesos rápidos para cargar archivos de texto plano, compilar visualmente (verificación de gramática), limpiar el entorno de trabajo y ejecutar (Correr) el código.

CONCLUSIÓN

La creación de un simulador de compilación completo y robusto ha sido el resultado del desarrollo de CODEMODE. La puesta en marcha pormenorizada de los analizadores léxicos, sintácticos y semánticos evidencia un entendimiento profundo de la manera en que los lenguajes de alto nivel procesan y convierten la información humana en órdenes ejecutables organizadas.

El entorno no experimenta cierres imprevistos gracias a la solución de errores lógicos avanzados (Edge Cases) y al mejoramiento en el manejo del almacenamiento de la memoria en la Tabla de Símbolos. Esto proporciona al usuario final una retroalimentación clara y didáctica, cumpliendo así con todos los objetivos propuestos para este proyecto de ingeniería.